Lazy: API decisions and tradeoffs

Decision record for Java architects — 25 September 2026 — branch `prototype/lazy` **.**

This document records the design developed in this discussion and embodied in the current specification of `java.util.Lazy` in this branch. It summarizes the decisions and their consequences; the source Javadoc remains the normative contract for this prototype. “Adopted” means adopted in this branch, not approved for the Java platform or committed to a release.

The starting point was the work associated with JDK-8336664, including `LazyConstant` and the `List.ofLazy` , `Map.ofLazy` , and `Set.ofLazy` factories, followed by Brian Goetz's email reproduced in the discussion. The email supplied the overall direction. Defects in the subsequent naming, typing, failure, memory, and collection decisions described here should be blamed on Per only. The pros and cons below are architectural assessments, not quotations or evidence of wider consensus.

The architectural direction

The adopted design is a non-generic, final utility class, `java.util.Lazy` , with generic factories. It packages common lazy-initialization and memoization use cases around three choices: the shape of access, the coordination/failure policy, and the location of storage. The computing function and policy are supplied at construction, keeping later access small and familiar.

Two principles from Brian's proposal drive the design. First, laziness has several legitimate meanings: initialization with irreversible side effects needs different coordination from a repeatable calculation. Second, ordinary users should not need to understand VM stability machinery to use the API correctly. Constant folding remains an optimization opportunity underneath a usable library abstraction.

The benefit of one factory class is a common vocabulary and one place to learn these differences. Its cost is a relatively substantial shared specification: the familiar return types do not express all the lazy semantics. The design deliberately covers selected useful combinations rather than every combination of coordinates, storage, failure policy, and memory ordering.

The central choices are standard `Supplier` / `Function` /collection interfaces, two modes named for what happens once, acquire/release publication, and explicit lookup authority for caller-owned field storage. The main costs accepted are semantics that must be learned from factory contracts, duplicate work or permanent failure depending on mode, and extra obligations when clients supply the storage field.

How the design evolved

Stage	Step taken	Step not taken
API organization	Followed Brian's final refinement toward one factory class and familiar access interfaces.	Did not expose the earlier family of `OfAbsolute` , `OfRelative` , `OfIndexed` , and `OfIndexedRelative` interfaces.

Stage	Step taken	Step not taken
Factory usability	Separated context access into `ofContext`; subsequently replaced the branch's `BoundFunction` with `Function`.	Did not retain overloaded `of(Mode, Supplier)` / `of(Mode, Function)` or a public custom context-access interface.
Policy naming	Evolved from the proposal's `InitMode` and the branch's `ONCE` / `RACE` terminology to `Mode.COMPUTE_ONCE` / `Mode.PUBLISH_ONCE`.	Did not make the public names depend on CAS or offer independent retry, exclusion, and fencing switches.
Memory contract	Replaced the earlier volatile-access guarantee with acquire/release publication, including recorded failures.	Did not promise a total ordering across different lazy values or extra synchronization from repeated access.
Field access	Added an exact field-type token, replaced caller sensitivity with an explicit `Lookup`, and permitted a superclass to declare a subclass accessor's field.	Did not suppress access checks, search inherited fields automatically, or allow arbitrary field updates.
Contract completion	Added `ofSet` and specified collection views, entries, traversal failures, streams, callback retention, composition, and execution context.	Did not treat ordinary interface defaults as sufficient to define these interactions.
Implementation	After the API reviews, brought implementations into line and added focused and general collection regressions.	Did not remove the earlier lazy APIs, settle their migration policy, or establish performance claims from benchmarks in this work.

Brian's final sketch used `BoundSupplier` and `InitMode`, still overloaded `of`, and did not include `ofSet`. The current design therefore follows that direction with substantive later refinements.

Current API surface

The following is a signature synopsis; bodies, annotations, and Javadoc are omitted. All factories are static members of `Lazy`.

```
public static <T> Supplier<T> of(Supplier<? extends T> computingFunction);
public static <T> Supplier<T> of(Mode mode,
                                 Supplier<? extends T> computingFunction);

public static <C, T> Function<C, T> ofContext(
        Mode mode, Function<? super C, ? extends T> computingFunction);

public static <C, T> Function<C, T> ofField(
        MethodHandles.Lookup lookup, Class<? super C> owner,
        String fieldName, Class<T> valueType,
        Function<? super C, ? extends T> computingFunction);

public static <T> List<T> ofList(
        Mode mode, int size, IntFunction<? extends T> computingFunction);
public static <K, V> Map<K, V> ofMap(
```

```
        Mode mode, Set<? extends K> keys,
        Function<? super K, ? extends V> computingFunction);
  public static <E> Set<E> ofSet(
        Mode mode, Set<? extends E> elementCandidates,
        Predicate<? super E> computingFunction);
```

The nested public types are `Mode`, with `COMPUTE_ONCE` and `PUBLISH_ONCE`, and the final error class `InitializationError`, with a public `InitializationError(String message)` constructor.

Factory	Unit of retained state	Access	Policy selection
`of`	One value per returned supplier	`get()`	Explicit mode, or `COMPUTE_ONCE` by default
`ofContext`	One value per returned function, across all contexts	`apply(context)`	Explicit mode
`ofField`	One value per receiver's designated field, shared by accessors using that field	`apply(receiver)`	Always `PUBLISH_ONCE`
`ofList`	One value per index	`get(index)` and ordinary list operations	One explicit mode, applied independently per element
`ofMap`	One value per copied key	`get(key)` and ordinary map operations	One explicit mode, applied independently per mapped value
`ofSet`	One Boolean membership result per copied candidate	`contains(candidate)` and ordinary set operations	One explicit mode, applied independently per candidate

Only `of(Supplier)` supplies a default mode. No public array factory or indexed-context shape is present. A list provides indexed access together with size, views, iteration, and streams; the price is a collection contract rather than a minimal coordinate interface. This is a surface-area choice, not a demonstrated claim of identical performance to an array.

Standard interfaces, overload resolution, and inference

Returning `Supplier` and `Function` avoids introducing new access interfaces. Replacing `BoundFunction<C,T>.get(C)` with `Function<C,T>.apply(C)` improves interoperability and gives users ordinary composition. The corresponding cost is semantic: `Function` alone does not tell the reader where storage lives or whether distinct arguments receive distinct results.

`ofContext` owns **one slot**, not a cache indexed by context. For example:

```
  Function<String, Integer> length =
        Lazy.ofContext(Lazy.Mode.PUBLISH_ONCE, String::length);
```

```
length.apply("abc");    // Installs 3.
length.apply("hello");  // Returns the retained 3.
```

An instance-derived property normally needs its own `ofContext` holder per instance. A static holder shared across instances would share one value. Supplying the context at access time permits a noncapturing callback; it does not eliminate the holder or guarantee allocation-free lambda execution. Neither context nor field access accepts a null context, even after initialization.

Renaming the context factory solves a different problem from generic inference. Explicit zero-argument and one-argument lambdas generally distinguish the original overloads, but overloaded or inexact method references—such as references to constructors with several arities—can make overload selection ambiguous. A destination result type does not generally rescue that selection. `ofContext` removes the competing factory overload and makes the access shape visible, at the cost of another method name.

The remaining `var` issue is ordinary inference: Java cannot infer `String` merely from `s -> s.length()` when no context type is otherwise supplied. A typed destination, an explicitly typed lambda, a suitable method reference, or explicit type arguments supplies that information. For example:

```
var length = Lazy.ofContext(Lazy.Mode.PUBLISH_ONCE,
                            (String s) -> s.length());
```

An additional `Class<C>` argument for `ofContext` was explored as an inference aid but was not adopted. It would make `var` examples easier to type, while adding an argument and another factory form despite there being no field type to validate in this case. This is a considered alternative, not a permanent rejection of any future convenience overload.

Composition retains the ordinary `Function` ordering: `compose` runs the preceding function on every application, and `andThen` runs the following function after every successful lazy access. Their results are not automatically cached. Composition itself records no additional initialization failure; nested lazy accesses still follow their own failure rules. This avoids silently changing familiar composition semantics, but requires callers to place expensive work inside the intended lazy boundary.

Two policies, named for their guarantees

Dimension	`COMPUTE_ONCE`	`PUBLISH_ONCE`
Intended use	Initialization, including side effects that must not repeat	Memoization where repeated computation is acceptable
Invocation count per slot	At most once; zero if never needed	Competing invocations and retries after failure are permitted
Competing access	Waits for the one invocation	May compute another candidate
Successful access	Returns the retained result	Returns the installed winner, including when its own candidate loses

Dimension	COMPUTE_ONCE	PUBLISH_ONCE
Failure	Permanent	Propagated to that caller; not retained
Principal benefit	Prevents duplicate initialization and repeated side effects	Avoids requiring all callers to wait for a designated initializing computation
Principal cost	Waiting, permanent failure, and possible dependency deadlock	Duplicate work, discarded results, and side effects that may repeat

`COMPUTE_ONCE` states what is limited; `PUBLISH_ONCE` states what is committed. They are more informative than `ONCE` / `RACE` and avoid encoding a particular instruction such as CAS. “Once” does not require eager execution. The installed result is consistent across successful accesses to one slot, but the identity or contents of the winning candidate need not be predictable across executions.

Initialization versus memoization is an intended-use distinction, not a static-versus-instance restriction. Either mode can serve either location where the chosen factory supports it.

The two policies bundle exclusion and failure handling. The benefit is a small choice set; the cost is that users cannot request, for example, serialized initialization with retry or concurrent computation with sticky failure. `PUBLISH_ONCE` is not a promise that the API or computing function is lock-free or wait-free. Discarded results are not closed or otherwise disposed of by `Lazy`.

Failure policy: preserve the Error category without retaining the original failure

The discussion explicitly chose to propagate an initial `Error` directly and keep its failure permanent in `COMPUTE_ONCE`. Subsequent observations remain in the `Error` hierarchy, using the dedicated `Lazy.InitializationError` rather than `NoSuchElementException`.

Computation outcome	Access executing the computation in `COMPUTE_ONCE`	Access observing its recorded failure in `COMPUTE_ONCE`	Failing access in `PUBLISH_ONCE`
Throws an `Error`	The same error, unchanged	A new `InitializationError`, no cause; original error class name when available	The same error, unchanged; retry remains possible
Throws another throwable	`NoSuchElementException` with that throwable as cause	`NoSuchElementException`, no cause; original throwable class name	The original throwable; retry remains possible
Returns null	Computation failure represented by a `NullPointerException`, wrapped as above	The recorded permanent failure	Direct `NullPointerException`; retry remains possible

Waiting callers count as subsequent accesses. The failure state does not retain the original throwable. A later `InitializationError` reports an earlier failure, not a fresh occurrence of the original error condition.

The benefit over wrapping everything in `NoSuchElementException` is that ordinary `catch (Exception)` handling does not accidentally absorb subsequent observations of an `Error`. Keeping only failure information also limits retention of exception stacks, application objects, and class loaders through the failure state. The costs are a new public error type and reduced diagnostics on later access: the original cause, stack, and exact error subtype are no longer available there. `NoSuchElementException` also has a second meaning in traversal, so a lazy failure must not be interpreted as end-of-iteration.

The Javadoc cites JLS 12.4.2. The analogy is deliberately limited: class initialization propagates an initial `Error` unchanged and later reports erroneous initialization with `NoClassDefFoundError`; `Lazy` uses its own failure types and storage rules. It does not import the whole class-initialization protocol.

Same-thread recursive access to an uninitialized `COMPUTE_ONCE` slot throws `IllegalStateException`. The outer computation may catch it and succeed; it becomes sticky only if it escapes that computation. Recursive `PUBLISH_ONCE` computation need not be detected. Cross-thread dependencies can deadlock in `COMPUTE_ONCE`. No general cycle detector was added.

In `PUBLISH_ONCE`, a failing computation still throws even if another thread has installed a value in the meantime. Substituting the winner for that failure would hide an outcome of user code; the chosen rule preserves exception propagation at the cost of some accesses failing after storage has become successfully initialized.

Memory consistency: publication, without repeated volatile synchronization

The adopted guarantee is release publication and acquire observation, for both successful values and permanent `COMPUTE_ONCE` failures. Actions before publication happen-before actions after an access observing that publication, including actions in a handler for a recorded failure.

Alternative	Benefit	Cost / status
Volatile-access contract	Stronger ordering, including a total order of volatile accesses	Stronger than the selected publication requirement; adds obligations to repeated access (e.g., prevents @Stable accesss). Replaced in this branch.
Acquire/release publication	Preserves visibility of initialization while allowing later reads to be elided when publication is preserved	Does not guarantee additional ordering or visibility for subsequent mutations, or total ordering across different lazy values. Adopted.
Plain access without publication guarantees	Potentially weaker access costs	Requires a weaker user model or external synchronization. Excluded from the offered modes.

The distinction for discarded computations was corrected during review:

- A successful access whose own candidate loses **does acquire the winner's publication**.
- That does **not publish the losing computation's own actions to other readers** through the lazy slot.
- Actions performed by a failed `PUBLISH_ONCE` computation are likewise not guaranteed to be published to other threads through that slot.

An atomic operation is not automatically sufficient for both directions. In particular, `compareAndExchangeRelease` reads its witness with plain semantics. The updated implementations acquire-read the installed value after a losing release exchange. This is an implementation technique satisfying the contract, not an instruction sequence required of every implementation.

The revised contract permits constant folding and elision of later value or failure reads only while preserving the applicable publication guarantee. It does not promise folding on every VM, equate all final-field chains with trusted fields, or make subsequent mutation of a returned object thread-safe. Lazily obtaining an array also does not make its elements lazy. Internal volatile operations remain permissible; changing the public contract did not mean removing every volatile field from the implementation.

Field storage: explicit authority and an intentionally narrower feature

`ofField` addresses the allocation-sensitive case identified in Brian's email: many cheap, short-lived objects whose caches are usually never read. A shared accessor uses storage already in each receiver, avoiding a separate lazy holder per receiver. The cost is loss of encapsulation of the storage protocol and a more demanding factory signature. Only `PUBLISH_ONCE` is offered for this shape.

Access-authority design	Pros	Cons	Decision
`@CallerSensitive`	Smaller call-site signature; caller identity is implicit	Authority is tied to invocation context; helper layers cannot simply carry the original caller's authority as an ordinary argument	Replaced
Explicit `MethodHandles.Lookup`	Authority is explicit, can be delegated through helpers, and follows established lookup/module rules	Extra argument; callers must obtain and deliberately pass an appropriately privileged capability	Adopted

A lookup is an access capability, not a bypass. The factory does not increase its permissions; the lookup class need not be the field owner. Field access is checked at factory creation. Invalid or inaccessible fields cause `IllegalArgumentException`; incompatible receivers or non-null computed values cause `ClassCastException` on access. A cross-package protected-field lookup can still restrict acceptable receivers to instances of its lookup class, even if another accessor has already initialized the field. This follows the established lookup capability model.

Several later changes make the field boundary precise:

Adopted rule	Benefit	Cost or obligation
The owner is `Class<? super C>` and must declare the field	Supports a subclass-specific callback over superclass storage	The caller must identify the declaring superclass; no inherited-field search
`Class<T> valueType` must exactly match the declared field's erased runtime type	Connects the accessor's value type to a checked field type; rejects mismatched tokens early	More verbosity; not a check of generic type arguments

Adopted rule	Benefit	Cost or obligation
Storage is a mutable, nullable instance reference field supporting the required atomic access	A null sentinel and one shared field give a simple storage protocol	No final, static, primitive, or null-restricted field storage
Non-null values may be preinitialized before safe receiver publication	Supports eagerly supplied values without computing again	The caller must safely publish the receiver and value
Later writes occur only through compatible `ofField` accessors	Preserves one retained value per receiver/field	Direct clearing or replacement violates the contract

For a `List<String>` field, the runtime check concerns `List`. A localized unchecked adaptation of `List.class` can supply `Class<List<String>>`, but the caller remains responsible for element-type correctness. Requiring exact erased-type agreement is therefore stronger than accepting any conveniently related token, but is not reified generic type safety.

Multiple functions may deliberately share the same field, even with different lookups or computing functions. Each receiver has one winner across those functions. The callbacks need not produce equal results; clients must accept any legitimately installed result, and no particular callback is guaranteed to execute. Each accessor retains its own lookup-imposed receiver restrictions. This enables cooperation between independently created accessors, while making their coordination and result compatibility a caller responsibility. The field is not a resettable cache.

Collections: retain ordinary shapes, specify the lazy boundary

The collection factories create unmodifiable collections. Every factory invocation supplies independent initially uninitialized storage; this does not require distinct result objects if callbacks return the same object. Views and map entries share their originating storage. Empty-instance reuse is permitted when construction has size zero, no keys, or no candidates; this permission does not turn a nonempty-candidate set that later filters to empty into shared lazy state.

Shape	What is fixed or copied eagerly	What is computed lazily	Main benefit / cost
List	Size and index domain	One element per index	Useful indexed interface and `RandomAccess` views; element-inspecting operations can trigger work or fail
Map	Copied key set	One value per key	Keys and size can be used without evaluating values; ordinary value-oriented operations can evaluate them
Set	Copied candidate set	One membership decision per candidate, retaining both true and false	Supports lazy filtering with stable membership; size and hash require all membership statuses

`ofSet` was added in this branch beyond Brian's final sketch. It computes membership of existing candidates, not new element objects or an unbounded stream.

List sublists and reversed views share original slots and compute with the original indices. Looking up an absent map key or an object outside a set's candidate domain does not invoke its computing function. An out-of-range list access fails before computation.

Map computations receive the **retained key from construction**, even when queried with a different but equal object. Set predicates receive the retained candidate on the same basis. The copies are shallow: the key/candidate objects themselves are retained, not cloned. This removes dependence on which equal query object arrives first. Its cost is that information present only on a query object is intentionally not an input to the cached computation. Input sets must use equality-compatible membership; changing key/candidate equality or hash codes while retained is outside the contract. Map and set iteration order is unspecified.

Copying a lazy input set can evaluate **the input's memberships** and propagate its failures. Construction does not thereby compute the output map's values or the output set's memberships. This distinction preserves snapshot semantics while making clear that accepting a lazy input is not a promise of a computation-free factory call.

The following stronger collection choices were made:

Decision	Pros	Cons
Null results are computation failures; null keys/candidates are rejected during construction	Clear distinction between uninitialized storage and a retained result	Absent computed values need an explicit representation such as `Optional`
Unsupported mutations always throw, even for no-ops, without invoking computing functions or supplied mutation callbacks	Predictable unmodifiability; `sort`, `replaceAll`, `removeIf`, and map updates cannot accidentally perform work	More overrides than permissive default methods; no “successful” empty mutation
Entries remain valid after traversal and share the original per-key storage	Entries can safely escape an iterator; later access observes the same result or permanent failure	Retaining one entry can retain the map and other unresolved state
Returned suppliers, functions, and collections are not serializable, including empty collections	Avoids defining serialization of callbacks, incomplete state, and sticky failures	Applications must choose their own materialized or domain-specific representation
Scalar `equals`, `hashCode`, and `toString` do not compute; collection content operations may compute	Keeps scalar diagnostics harmless while retaining ordinary collection meanings	Collection logging, hashing, equality, or copying can run user code and fail

Obtaining map entries or their keys does not itself compute mapped values. Reading an entry's value does; `Map.Entry.copyOf` therefore may compute or report a failure while producing its detached copy. Unmodifiable collections can still contain mutable values. In particular, a lazy map's structural hash must reflect current value hashes rather than be permanently cached merely because its storage locations never change.

Traversal and streams: failures do not silently remove data

When the traversal itself encounters a lazy computation failure, the pending value or candidate is not consumed. A failed list-iterator `next` or `previous` leaves its cursor unchanged. Set iterators may advance past candidates already known to be absent, but keep a failing membership pending. The rules cover bulk traversal, `hasNext` / `hasPrevious`, and spliterators; a failed source access in `tryAdvance` does not invoke its action. A split that fails because of lazy computation must preserve the original's remaining coverage.

The benefit is that retry in `PUBLISH_ONCE` cannot silently skip data, while `COMPUTE_ONCE` repeatedly reports the same slot's recorded failure. The cost is extra traversal state and implementations beyond ordinary defaults. This is not transactional rollback: earlier values, memberships, and failures remain retained, and elements already delivered remain consumed.

Failures thrown by a traversal action follow the ordinary interface's continuation rules, including when that action makes its own failing lazy access. Such a failure must not be mistaken for failure of the traversal's own source access. More generally, user callbacks outside the computing function do not themselves create an additional initialization failure; lazy accesses inside them retain their own rules.

Creating streams or spliterators and querying their metadata do not compute values or report stored computation failures. Source spliterators, including those produced by splitting, report `NONNULL` and `IMMUTABLE`; lists and views additionally report `ORDERED`, `SIZED`, and `SUBSIZED`. Map key/entry views report `DISTINCT`, `SIZED`, and `SUBSIZED`; value views report the latter two. Lazy sets report `DISTINCT` and need not report an exact size before it is known without evaluation.

The design does not promise minimal evaluation by every traversal, split, or parallel operation. Short-circuiting can compute more than the final result consumes. Conversely, a sized `count()` can avoid element computation altogether, even when some element has a recorded failure. This preserves stream optimization freedom. A terminal operation such as `count()` is not evidence that every lazy value has been evaluated; validation must explicitly traverse or access the relevant values. The current implementation's non-computing range splits are stronger than the minimum public guarantee and are not a new API commitment.

Execution, retention, and lifetime

Computations run synchronously on the accessing thread. `COMPUTE_ONCE` coordinates one slot; a shared collection callback may still execute concurrently for different slots. Waiting is uninterruptible and preserves an interrupt already present or received during the wait, on normal or abrupt completion. A computing function's own interruption handling is not constrained by that waiting rule. Cancellation, timeouts, and asynchronous execution are not provided.

This keeps access and exception propagation direct and avoids an executor policy. It also means that a blocking computation blocks its caller, and potentially other callers waiting for its result. Neither mode provides a general liveness guarantee.

Storage-owning factories strongly retain the computing function while any slot remains unresolved, and release the invocation reference when all slots have either a result or permanent failure. Mixed successful and failed slots count as terminal. A view's lifetime is assessed against the whole original storage, including slots outside that view. `ofField` instead retains its callback for the accessor's lifetime because new receivers can always arrive.

This rule bounds retention after completed initialization while ensuring that unresolved slots remain computable. It cannot guarantee garbage collection: values, keys, candidates, in-flight computations, and other application references may still retain the callback. `ofContext` does not otherwise retain its context, and `ofField` does not otherwise retain receivers; callbacks or results can themselves contain references.

Records and Valhalla: supported patterns and limits

The discussion tested the derived-property use case for `record Point(int x, int y)`. That exact record cannot declare extra instance fields for `radius` and `phi`; see JLS 8.10.2. A static `ofContext` would incorrectly share one slot across points. The branch's field-backed example therefore uses an ordinary class. Adding holders as record components would change its representation and generated equality; no transparent record cache facility or new language feature was implemented.

For the Valhalla value-class model examined here, a final field can reference a mutable lazy holder, and `ofContext` with a noncapturing callback avoids capturing incompletely initialized `this`. Direct `ofField` storage is unsuitable because value-class instance fields are final. The important cost is equality: holder references participate in value-object `==` and inherited `Object.equals`, so objects with otherwise equal application data may compare unequal. Custom `equals` / `hashCode` can define logical equality but cannot change `==`. These points concern the Valhalla model, whose draft specification remains a separate language-development effort.

The benefit is a workable library pattern without adding language machinery. The costs are holder storage and possible equality consequences. The API does not make all capturing lambdas illegal, guarantee allocation-free value-class caching, or supply an indexed-relative collection shape whose ordinary `List.get` accepts an owning value object.

Alternatives outside the adopted scope

The distinctions in this table matter: absence from the current surface is not proof of permanent rejection.

Alternative	Status in this work	Reason or unresolved tradeoff
Lazy fields / cached methods as new language features	Discussed in Brian's background; not implemented here	Static and instance use cases do not divide cleanly; records and value classes add separate constraints
Public stability annotation or general stable VarHandle/MethodHandle toolkit	Not exposed by this API	The high-level entry point targets common usage; possible lower-level facilities remain separate work
Full coordinate family, including indexed-relative access	Present in earlier sketches; absent from the adopted surface	A smaller API loses some noncapturing context-plus-index use cases
Context class-token overload	Explored, not adopted	Better inference cues versus extra signature surface
Primitive specializations	Not provided; no permanent rejection recorded	Boxing and storage costs remain; primitive-specific APIs would enlarge the surface
Null as a successful lazy result	Not offered	Non-null retained values simplify the contract; absence needs another representation
Reset, invalidation, eviction, or per-argument memoization	Not provided	These require a lifecycle beyond one retained result per storage location

Alternative	Status in this work	Reason or unresolved tradeoff
Explicit initialization-state query or non-computing peek	Not provided	Standard return interfaces expose access, not a new lifecycle protocol
Serialized retry, independent policy flags, `COMPUTE_ONCE` field access	Not provided	The design deliberately stops short of the full policy/storage product
Cancellation, timeouts, executor-backed evaluation	Not provided	No scheduling or task-management policy is added to lazy access
Removal or deprecation of the earlier lazy APIs	Not performed	Migration and compatibility policy require a separate platform decision

Implementation and validation status

The implementation work followed the specification reviews. It updated scalar, field, and collection publication; retained stable storage opportunities; added failure-preserving traversal and views; preserved original storage through map entries and key views; corrected equality exception propagation; and preserved publication through cached set size/hash results. Existing legacy factory behavior was kept on its existing paths.

The final validation sequence built `java.base` and passed 13 targeted jtreg test descriptions, comprising 821 successful JUnit invocations plus non-JUnit checks. Coverage included the new Lazy suites, legacy lazy-list/map/set tests, a negative compilation test, lookup/module tests, and MOAT collection checks extended to both new modes. One new test file initially needed a generic assertion typing correction and then passed on a focused rerun; the combined results are all passing.

That evidence supports the tested implementation behavior, not a proof of all memory-model executions, measured allocation savings, or a performance comparison across VMs. The outstanding platform decisions are adoption and release sequencing, the relationship to the earlier APIs, and whether workload evidence warrants any additional shapes or specializations. The current branch records the smaller API and its detailed contracts without committing to those extensions.